



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 人工智能 (实验)
实验名称: 实验三强化学习
实验性质: 综合设计型
实验学时: 2 地点: T2612
学生姓名: 刘睿
学生学号: 220110720
学生班级: 计科 7 班
授课教师: _____

实验与创新实践教育中心制

2025 年 3 月

一、实验环境

描述操作系统、开发环境（CPU\GPU）、使用的库等。

使用 Google Colab 环境

```
1 !uname -a

Linux 6e0bce01463a 6.1.123+ #1 SMP PREEMPT_DYNAMIC Sun Mar 30 16:01:29 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux


1 !nvidia-smi

Fri May 23 04:46:11 2025
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
+-----+-----+
| GPU  Name            Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC | |
| Fan  Temp            Perf          | Pwr:Usage/Cap |      Memory-Usage   | GPU-Util  Compute M. |
|                                           MIG M.         |                      |
+-----+-----+
|  0   Tesla T4                Off   | 00000000:00:04:0 Off |                    0 | |
| N/A   56C            P0              | 26W / 70W     | 174MiB / 15360MiB   |      0%    Default  |
|                                           N/A             |                      |
+-----+-----+

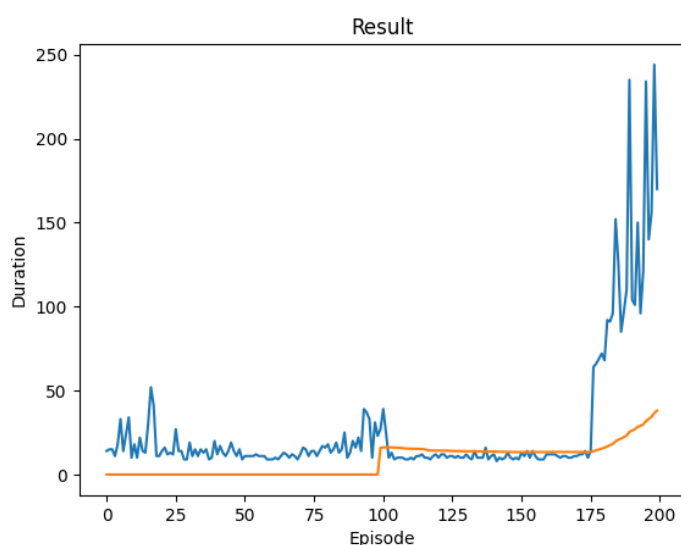
+-----+
| Processes:                        |
| GPU   GI    CI          PID    Type    Process name          GPU Memory |
| ID    ID    ID              |              |                     |      Usage |
+-----+-----+
|
```

二、实验过程和结果分析

2.1 初始代码运行结果

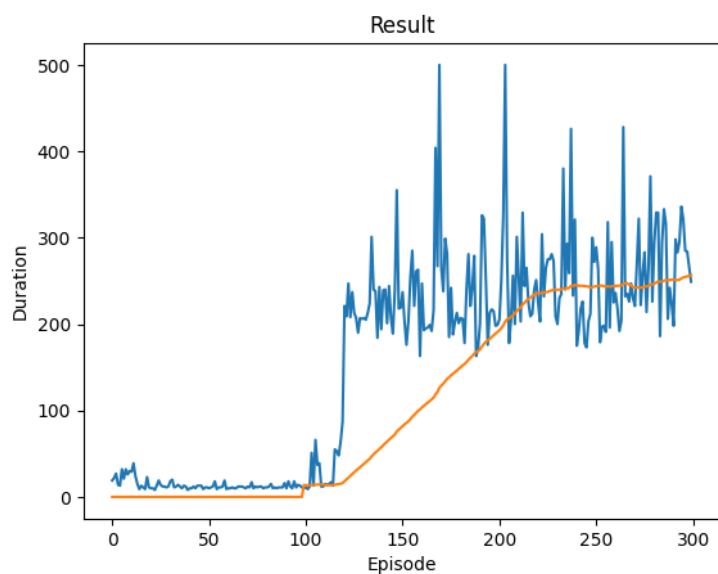
修改相关参数，运行 `reinforcement_q_learning.ipynb`，使得训练收敛，并将结果截图，并阅读代码解释图中黄线的含义。

未经修改直接运行后的结果如图：



可以看到，前面的训练速度有点慢了，在 175 回合后突然学会了策略，表现显著提高，但由于我们只设定了 200 次迭代，结果尚未稳定训练就结束了，尚未达到收敛。所以，我们可以考虑增加 100 次迭代作为一次尝试

在 300 次迭代的设定下，我们重新运行程序，最后得到结果如下：



可以看到的是，蓝线有两次达到了 500 大关。最后和黄线的趋于一致，在黄线附近浮动，黄线此时已经变得平缓，虽说不是一定完全收敛了，但是暂时就这样吧

至于这个黄线是什么，我们可以查看 `plot_durations()` 函数

```
53 def plot_durations(show_result=False):
54     plt.figure(1)
55     durations_t = torch.tensor(episode_durations, dtype=torch.float)
56     if show_result:
57         plt.title('Result')
58     else:
59         plt.clf()
60         plt.title('Training...')
61     plt.xlabel('Episode')
62     plt.ylabel('Duration')
63     plt.plot(durations_t.numpy())
64     # Take 100 episode averages and plot them too
65     if len(durations_t) >= 100:
66         means = durations_t.unfold(0, 100, 1).mean(1).view(-1)
67         means = torch.cat((torch.zeros(99), means))
68         plt.plot(means.numpy())
69
70     plt.pause(0.001) # pause a bit so that plots are updated
71     if is_ipython:
72         if not show_result:
73             display.display(plt.gcf())
74             display.clear_output(wait=True)
75         else:
76             display.display(plt.gcf())
```

可以看得到，黄线的逻辑是窗口大小为 100 的滑动平均，至于为何前 100 Episode 黄线值为 0，是因为我们主动填充了前面 99 个 0，直到第 100 Episode 我们有 100 个数据来计算滑动平均的时候，我们才计算出第一个真实的滑动平均结果

2.2 优化代码及运行结果

可从神经网络结构的优化、超参数调优、优化经验回放区、奖励函数的设计、探索策略的设计等方面着手，挑选 4 个方向进行优化，并分析对比结果。

1. 优化一

(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

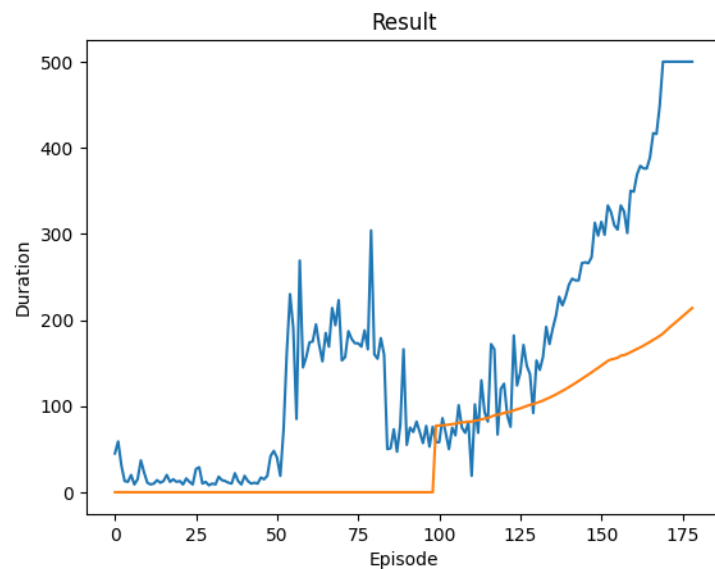
优化一我们选择对神经网络结构进行优化，我们增加层数和节点数，以此来提升表达能力，适合学习更加稳定的策略

```

1  class DQN(nn.Module):
2
3      def __init__(self, n_observations, n_actions):
4          super(DQN, self).__init__()
5          # self.layer1 = nn.Linear(n_observations, 128)
6          # self.layer2 = nn.Linear(128, 128)
7          # self.layer3 = nn.Linear(128, n_actions)
8          self.layer1 = nn.Linear(n_observations, 256)
9          self.layer2 = nn.Linear(256, 256)
10         self.layer3 = nn.Linear(256, 128)
11         self.layer4 = nn.Linear(128, n_actions)
12
13         # Called with either one element to determine next action, or a batch
14         # during optimization. Returns tensor([[left0exp,right0exp]...]).
15     def forward(self, x):
16         x = F.relu(self.layer1(x))
17         x = F.relu(self.layer2(x))
18         x = F.relu(self.layer3(x))
19         return self.layer4(x)

```

(2) 运行结果截图



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

可以看到的是，在优化神经网络结构后，我们在前面 100 个 Episode 经历一次回撤后，Duration 开始稳步上升，在 70 个 Episode 左右就达到了 500 的目标 Duration，并稳定获得 10 次成功之后在第 179 Episode 触发早停机机制。以上说明我们对神经网络结构优化的策略取得了相当良好的效果

2. 优化二

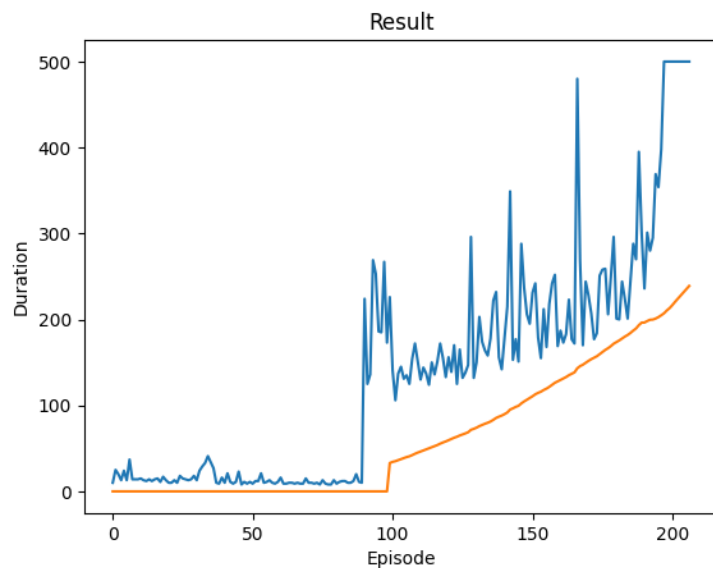
(1) 优化代码描述

截图代码粘贴于此，并简单描述优化内容。

优化二我们从超参数调优着手。我们的目标是要突破初始结果的瓶颈，我们尝试几个方案后，最后在这个方案上取得了良好表现。其中减小 EPS_DECAY，增加 LR 是期待能够加速策略的形成。减小 BATCH_SIZE 和增大 TAU 是为了加快更新的速度

```
9  BATCH_SIZE = 64      # 128 -> 64
10  GAMMA = 0.99
11  EPS_START = 0.9
12  EPS_END = 0.05
13  EPS_DECAY = 500      # 1000 -> 500
14  TAU = 0.01           # 0.005 -> 0.01
15  LR = 3e-4            # 1e-4 -> 3e-4
```

(2) 运行结果截图



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

对比初始结果，我们突破了一开始均值在 250 左右的瓶颈，并在第 207 Episode 处触发了早停。这个结果已经远超我的预期了，我本不对超参数优化抱有期待，觉得难以突破简单模型的桎梏，但最后实际却取得了相当不错的结果

3. 优化三

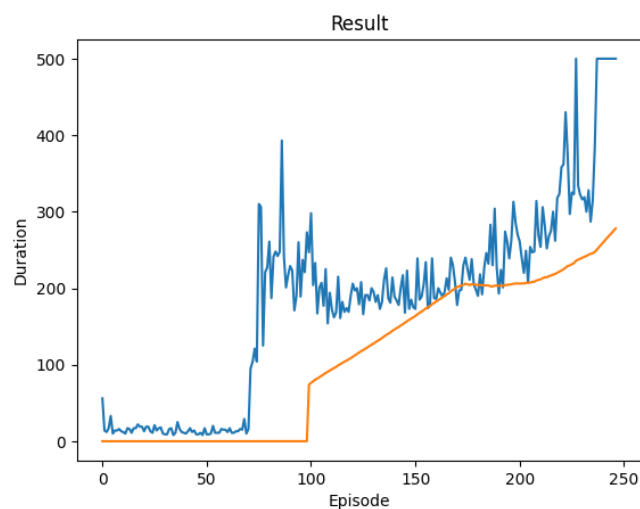
(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

优化三我们选则 PER，用以替代原始的均匀随机经验采样机制。该机制通过追踪每条 transition 的 TD-error（时序差分误差），动态调整其被采样的概率，优先学习“误差大、价值高”的经验样本，从而提升收敛速度与策略稳定性

```
1 import numpy as np
2
3 class PrioritizedReplayMemory():
4     def __init__(self, capacity, alpha=0.6):
5         self.capacity = capacity
6         self.buffer = []
7         self.priorities = np.zeros((capacity,), dtype=np.float32)
8         self.pos = 0
9         self.alpha = alpha
10
11     def push(self, *args):
12         """Store a new transition with max priority"""
13         max_prio = self.priorities.max() if self.buffer else 1.0
14         transition = Transition(*args)
15         if len(self.buffer) < self.capacity:
16             self.buffer.append(transition)
17         else:
18             self.buffer[self.pos] = transition
19             self.priorities[self.pos] = max_prio
20             self.pos = (self.pos + 1) % self.capacity
21
22     def sample(self, batch_size, beta=0.4):
23         if len(self.buffer) == self.capacity:
24             prios = self.priorities
25         else:
26             prios = self.priorities[:self.pos]
27
28         probs = prios ** self.alpha
29         probs /= probs.sum()
30
31         indices = np.random.choice(len(self.buffer), batch_size, p=probs)
32         samples = [self.buffer[i] for i in indices]
33
34         total = len(self.buffer)
35         weights = (total * probs[indices]) ** (-beta)
36         weights /= weights.max()
37         weights = torch.tensor(weights, dtype=torch.float32, device=device)
38
39         return samples, indices, weights
40
41     def update_priorities(self, indices, td_errors, epsilon=1e-5):
42         for idx, td_err in zip(indices, td_errors):
43             self.priorities[idx] = abs(td_err) + epsilon
44
45     def __len__(self):
46         return len(self.buffer)
```

(2) 运行结果截图



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

对比初始结果有了良好的改进，并在第 247 Episode 触发早停。整体上升过程较为平稳，在经验池累积起来之后基本处于稳步上升的状态，浮动程度较小

4. 优化四

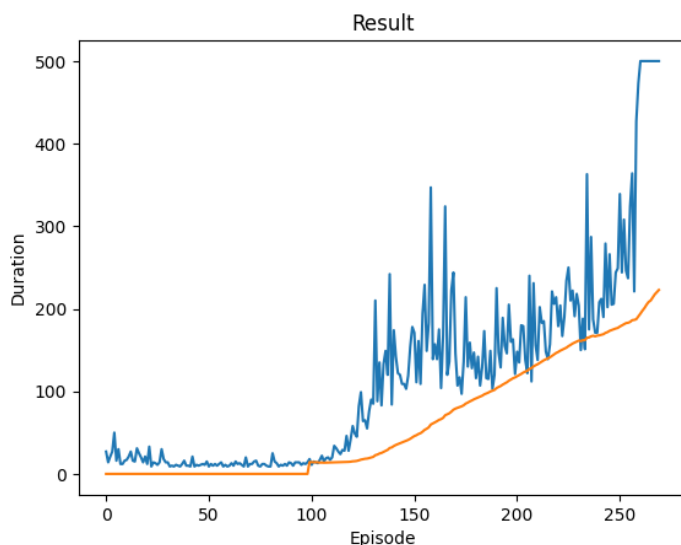
(1) 优化代码描述

代码截图粘贴于此，并简单描述优化内容。

为提升策略学习效率，我们设计了一个基于状态信息的奖励函数 `compute_shaped_reward()`，并将其与环境原始奖励叠加使用。该函数考虑了智能体在 `CartPole-v1` 环境中的核心状态变量：位置、速度、角度和角速度，以提供更加细粒度的学习信号

```
1 def compute_shaped_reward(observation):
2     x, x_dot, theta, theta_dot = observation
3
4     x_limit = env.unwrapped.x_threshold
5     theta_limit = env.unwrapped.theta_threshold_radians
6
7     r1 = (x_limit - abs(x)) / x_limit
8     r2 = (theta_limit - abs(theta)) / theta_limit
9     r3 = 1 - abs(x_dot) / 2.0      # 平滑惩罚，速度越小越好
10    r4 = 1 - abs(theta_dot) / 2.0   # 杆子角速度越小越好
11
12    shaped = 0.25 * r1 + 0.25 * r2 + 0.25 * r3 + 0.25 * r4
13    return max(shaped, 0) # 避免负值
```

(2) 运行结果截图



(3) 对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

在第 270 Episode 触发早停，整体上升较为平缓，在 250 回合后突然习得优秀策略，并逐渐稳定下来